优化：病态 Hessian、随机梯度下降与动量法

读前须知

- **前置**：梯度下降、Hessian 矩阵与条件数的概念。
- **这一篇的位置**：训练过程的动力学。同样的模型与数据，换一个优化器，收敛速度可能差几十倍。
- **读完能做什么**：用 Hessian 的条件数解释“病态”为什么让梯度下降变慢；说清小批量 SGD 的噪声为何反而有益；比较动量法与自适应学习率的适用场景。

一、深度优化难在哪里

1.1 非凸性

除了正则化之外，优化也是深度学习需要解决的一个核心问题。由于深度神经网络中的隐藏层数目较多，因而将整个网络作为一个整体进行优化是非常困难的事情，需要花费大量的时间和计算力。出于效率和精确性的考虑，在深度学习的优化上需要使用专门的技术。

出于可解性的考虑，传统机器学习算法往往会小心翼翼地选择代价函数和优化条件，将待优化问题转化为容易求解的凸优化问题。但在神经网络，尤其是在深度神经网络中，更一般的非凸情况是不可避免的，这就给深度学习中的优化带来很多额外的挑战。

凸优化之所以好对付，是因为它有两条性质：任何局部极小都是全局极小；最优解可以用多项式时间求得。深度网络的损失函数把多层非线性变换复合起来，这两条性质都不再成立，于是问题从“怎么算得快”变成了“即使算出来，算到的是不是想要的”。

1.2 三类障碍

深度优化的困难可以归纳成三个来源：

1. **病态**：损失曲面的不同方向曲率差异极大，梯度下降在某个方向上抖动，在另一个方向上几乎不动；
2. **局部极小**：存在大量局部最优点，搜索可能停在一个次优的位置；
3. **鞍点**：梯度为零但不是极值点，算法在这里陷入停滞。

后两个问题的根源都在高维空间里，而高维空间的几何与低维直觉差别很大。下面逐一分析。

二、病态 Hessian

2.1 条件数

当待优化的代价函数的输入是 n 维向量时，其二阶导数就包含针对不同变量分别求偏导所得到的很多分量，将这些分量按顺序排列可以得到 **Hessian 矩阵**：

$$H_{ij} = \frac{\partial^2 f}{\partial x_i \, \partial x_j}$$

Hessian 矩阵描述损失曲面在某点附近的曲率。它的特征值对应各个主方向上的曲率大小，特征向量指向这些主方向。

在线性方程 Ax=b 中，当系数矩阵 A 的微小扰动会给解集 x 带来较大幅度的波动时，这样的矩阵就被称为**病态矩阵**（ill-conditioned matrix）。病态矩阵是科学计算不愿打交道的对象，因为数值精度导致的不可避免的舍入误差可能会给输出带来巨大的偏离，正所谓“差之毫厘，谬以千里”。

衡量病态程度的是**条件数**，定义为最大特征值与最小特征值之比：

$$\kappa(H) = \frac{\lambda_{\max}}{\lambda_{\min}}$$

条件数为 1 时曲面是各向同性的球面，条件数很大时曲面像一个极扁的峡谷，不同方向上的尺度相差几个数量级。

补充说明：条件数之所以直接决定收敛速度，可以从二次型损失看出来。在梯度下降下，误差沿各特征方向独立衰减，衰减因子约为 (1−ηλᵢ)。若 η 取 1/λ*max 以保证最大曲率方向不发散，则最小曲率方向的衰减因子只有 1*−λmin/λ_max，每步只减少一点点。于是迭代次数大致与 κ 成正比。在深度网络中，Hessian 的最小特征值常常接近零——损失曲面在某些方向上几乎是平的——条件数可以大到上万，这就是训练缓慢的直接原因。预条件、批归一化、动量，做法虽不同，共同目标都是“把各方向的尺度拉平”。

2.2 对梯度下降的影响

在神经网络的训练中，Hessian 矩阵的病态问题非常常见，甚至无论优化问题是否具有凸优化的形式，病态的 Hessian 矩阵都会出现。在神经网络的训练中，病态矩阵的影响体现在梯度下降的不稳定性上。当应用随机梯度下降解决优化问题时，病态矩阵对输入的敏感性会导致很小的更新步长也会增加代价函数，使学习的速度变得异常缓慢。

三、局部极小与鞍点

3.1 隐藏单元不可辨认

深度神经网络面临的另一个挑战是局部极小值的问题。凸优化问题的数学特性保证了局部极小值和全局最小值之间的等价关系。因而在优化一个凸问题时，任何形式的临界点都可以看成是可行解。而在神经网络，尤其是深度模型中，代价函数甚至会具有不可列无限多个局部极小值，这显然会妨碍对全局最小值的寻找，导致搜索陷入局部最优的陷阱中。

神经网络之所以会具有这么多局部极小值，原因在于**隐藏变量的不可辨认性。**如果将神经网络中的几个隐藏神经元及其所有系数调换的话，得到的新模型和原始模型之间是等价的。假定深度模型中包含 m 个隐藏层，每个层中又都有 n 个神经元，那么隐藏单元的排列方式就会多达：

$$(n!)^m$$

这么多神经网络的变体是没法相互区分的，因而它们都有相同的局部极小值。

这类由对称性导致的极小值并不可怕，因为它们的函数值完全相同，落到哪一个都是同一个解。真正麻烦的是函数值不同的那些局部极小。

3.2 鞍点与牛顿法

除了局部极小值外，另一类在优化中不受欢迎的点叫做**鞍点**（saddle point）。鞍点是梯度为 0 的临界点，但它既不是极大值也不是极小值。从函数图像上看，多变量函数的鞍点在一个方向上向上弯曲，在另一个方向上则向下弯曲，从而形成了类似马鞍的形状。

由于牛顿法的目标是寻找梯度为零的临界点，因而会受鞍点的影响较大，高维空间中鞍点数目的激增就会严重限制牛顿法的性能。

> **新手提示：局部极小没有那么可怕**

> “深度网络会陷入局部最优”是最常被提及的担忧，但在高维空间里，这个担忧的优先级比想象中低。一个点要成为局部极小，需要在所有维度上曲率都为正；而维度成千上万时，所有方向同时向上弯的概率极低。随机取一个临界点，更可能是在少数方向上向上、在多数方向上向下的鞍点。鞍点为优化提供了大量可以继续下降的方向，只要算法不被零梯度卡住（比如动量和噪声就能帮它滚下去），它总能继续走。实践中，训练卡住更多是学习率、初始化或数据的问题，而不是“恰好落在一个坑里”。

补充说明：在高维空间中，鞍点比局部极小常见得多，这一点可以用随机矩阵理论粗略说明。若把 Hessian 的每个特征值看作独立同分布的随机变量，则“所有特征值同号”的概率随维度 n 指数衰减，而“有正有负”的概率趋于 1。所以维度越高，随机临界点越可能是鞍点。这也解释了一个反直觉的现象：深层网络反而比浅层网络更容易训练出好结果，因为它有更多逃逸方向。

四、随机梯度下降

4.1 从全量到小批量

虽然存在着这样那样的潜在问题，但深度学习采用的依然是传统优化方法及其改进，这是由于隐藏层的未知特性使设计有针对性的优化方法变得非常困难。

机器学习中的最优化是令给定的期望风险函数最小化，而期望风险又可以用训练集上的经验风险 Q(w) 代替：

$$Q(w) = \frac{1}{n}\sum_{i=1}^{n} Q_i(w)$$

训练集上的经验风险等于所有样本风险函数 $Q_i(w)$ 的均值。当训练集的数据量较大，并且经验风险又难以用简单的解析式来表示时，计算所有梯度的求和就会变成一个复杂的计算任务，消耗大量的计算资源。

随机梯度下降法（stochastic gradient descent）就是在传统机器学习和深度神经网络中都能发挥作用的经典算法。

4.2 更新公式与学习率

随机梯度下降法是原始梯度下降法的一种改良。如果把求解最优化问题想象成爬山，那么随机梯度下降法就是每走一步就换个方向。

为了节省每次迭代的计算成本，随机梯度下降在每一次迭代中都使用训练数据集的一个较小子集来求解梯度的均值，这在大规模机器学习问题中，特别是深度学习中非常有效。在给定学习率 η 和前一轮的参数估计值 w 时，随机梯度下降法的每一轮迭代执行以下操作：从训练集中随机抽取 n' 个样本，在这个样本子集上计算梯度估计值，再根据计算出的结果更新参数：

$$w \leftarrow w - \eta \cdot \frac{1}{n'}\sum_{i=1}^{n'} \nabla_w Q_i(w)$$

以上迭代过程一直重复到 Q(w) 取得最小值或迭代次数达到最大值为止。

学习率 η 是随机梯度下降法中的重要参数，它通常被设置为随着训练次数的增加而逐渐减小。这一选择体现的是训练时间和优化性能之间的折中。相对于让所有训练样本的风险函数最小化的原始梯度下降法，随机梯度下降是让每几个样本的风险函数最小化，虽然不是每次迭代得到的结果都指向全局最优方向，但大方向终归是没有错的，其最终的结果往往也在全局最优解附近。

> **新手提示：“随机”指的是采样，不是结果**

> 随机梯度下降的“随机”说的是每次只从训练集中随机抽一小批样本来估计梯度，而不是说结果不可靠。单次梯度估计确实带噪声，方向可能与真实梯度有偏差；但由于噪声的期望为零，多步累计下来，参数仍然沿着总体下降的方向移动。噪声甚至是有益的：它相当于给优化过程加了扰动，能帮助算法越过鞍点和尖锐的局部极小。批量越小，噪声越大，逃逸能力越强；批量越大，梯度越准，但计算开销也越大。这就是批量大小需要权衡的原因。

补充说明：学习率调度是深度学习中最影响结果的超参数之一。常见策略有三种：**阶梯衰减**，在固定轮次把学习率乘 0.1；**余弦退火**，按余弦曲线从初始值降到接近零；**预热**，前几百步从很小的值线性升到目标值，避免训练初期因梯度不稳定而发散。预热在大批量训练与 Transformer 中几乎是必须的，因为初始阶段参数随机、梯度方向混乱，直接上大学习率容易把模型推入坏区域。经验上，学习率设得比“能收敛的最大值”略小一点，往往比精细调度更有效。

五、降噪：让梯度更准

5.1 动态采样

在随机梯度下降法的基础上进行改进可以得到其他的优化方式，改进的手段主要有两种：一种是随机降低噪声，另一种是使用二阶导数近似。

在应用中，随机梯度下降会受到噪声的影响。当学习率固定时，噪声会阻止算法的收敛；而当学习率逐渐衰减时，噪声也会将收敛速度压低到次线性水平。降噪方法正是为了抑制噪声的影响应运而生，降噪的方式既包括提升单次梯度估计的精度，也包括提升迭代过程的精度，常用的算法包括动态采样、梯度聚合和迭代平均三类。

动态采样和梯度聚合两类方法是通过使用固定的步长来获得线性的收敛速度，进而实现降噪。动态采样方法通过逐渐增加梯度计算中使用的子集容量来实现降噪，随着优化过程的进行，基于更多样本得到的梯度估计也就越来越精确。

5.2 梯度聚合

梯度聚合方法则将先前迭代中得到的梯度估计存储下来，并在每次迭代中对这些估计中的一个或多个进行更新，再通过将搜索方向定义为之前轮次中梯度估计的加权平均，来改善搜索方向的准确性。

5.3 迭代平均

和前面两种方法相比，迭代平均方法不是通过对梯度估计求平均，而是对每次迭代得到的参数结果求平均来实现降噪。迭代平均方法可以在保证算法收敛性的前提下提升学习率，从而缩短收敛时间。这种方法在思想上更接近原始的随机梯度下降法，虽然它的收敛速度依然保持在次线性水平，却可以降低结果的方差，从而有效地对抗过拟合问题。

补充说明：批量大小与学习率需要联动调节。常用的经验法则是**线性缩放**：批量扩大 k 倍，学习率也扩大 k 倍。理由是批量增大使梯度估计的方差降为原来的 1/k，允许更大地迈步；同时每轮迭代处理了 k 倍的样本，为了保持相同的“每样本更新量”，学习率也要相应放大。这条法则在批量达到几千以内基本有效，再大就会出现收益递减，此时需要配合预热或分层自适应学习率。反过来，如果显存不够只能用小批量，就要把学习率调小，否则噪声过大导致训练震荡。

六、二阶信息

6.1 牛顿法

要提升随机梯度下降法的性能，还可以通过使用二阶导数近似的信息来抑制高度非线性和病态目标函数的不利影响。在梯度下降法中应用的一阶导数不具有线性不变性，二阶导数的引入可以解决这一问题。另一方面，使用二阶导数意味着在使用泰勒展开近似目标函数时，二阶项的引入可以加快对最值点的逼近。

牛顿法在当前位置做二次近似，直接跳到该二次函数的极小点：

$$\theta \leftarrow \theta - H^{-1}\nabla_\theta J(\theta)$$

这一步在二次损失上一步到位，在一般的凸函数上也能获得二次收敛速度，远快于梯度下降的线性收敛。

6.2 拟牛顿与高斯牛顿

总而言之，二阶导数近似方法主要的优势在于提升收敛速度。二阶导数近似方法都可以看成对传统的牛顿法的改进，具体方法包括拟牛顿法、高斯牛顿法和无 Hessian 牛顿法等，它们的共同点是避开 Hessian 的显式计算，用梯度差或雅可比矩阵来近似曲率信息。

补充说明：牛顿法在深度学习中很少直接使用，原因很实在。参数有 n 个时，Hessian 是 n×n 矩阵，存储就要 $O(n^2)$，求逆更是 $O(n^3)$。一个千万参数的模型，Hessian 需要 10^{14} 个元素，完全无法承受。拟牛顿法（如 BFGS、L-BFGS）通过逐步近似逆矩阵避开显式求逆，L-BFGS 只保存最近若干步的梯度差与参数差，内存降到 O(n)，在小规模问题上表现很好。但在小批量训练下，梯度本身带噪声，二阶近似会被噪声污染，效果反而可能不如带动量的 SGD。这也是为什么大规模深度学习的主流依然是 SGD 家族，二阶方法更多出现在全批量可计算、参数规模较小的场景。

> **新手提示：优化器不是越新越好**
>
> 新算法通常在特定基准上胜过旧算法，但换一个任务未必。带动量的 SGD 在图像分类上经常与自适应方法打平甚至略胜，而且最终精度往往更好；自适应方法（Adam 系列）在语言模型、稀疏梯度和需要快速启动的场景中优势明显。选择优化器的实用建议是：先固定学习率与批量大小，用带动量的 SGD 跑一个基线，再用 Adam 跑同样的设置对比。如果两者差不多，优先选 SGD，因为它需要的显存少、超参少、对正则化更友好。另外，换优化器时学习率不能沿用，Adam 的典型学习率比 SGD 小一个数量级。

七、动量类方法

7.1 动量

除了随机降噪和二阶导数近似这两大类主要方法之外，还有一些自立门户的算法也能够对随机梯度下降做出改进，典型的例子包括动量方法（momentum）、加速下降方法（accelerated gradient descent）和坐标下降方法（coordinate descent）。

动量方法引入了超参数 v 作为速度，它代表参数在参数空间上移动的方向和速率，移动参数的力则是负梯度：

$$v \leftarrow \mu v - \eta\nabla_\theta J(\theta), \qquad \theta \leftarrow \theta + v$$

其中 μ 是动量系数，通常取 0.9 左右。速度 v 的引入就会让之前的移动产生累加效应，累加的效果则体现在学习率上。如果参数在一个下降较快的方向上移动，那么就迈开大步向前冲，对应着学习率的增加；如果参数在忽左忽右地兜圈子，那就找一条中间路线向前走。这样一来，算法就不太会陷入到局部最优中而无法抽身了。

补充说明：动量对病态问题有直接的缓解作用。在狭长的峡谷地形中，梯度在陡峭方向上分量很大、在平缓方向上分量很小，普通梯度下降会在谷壁之间来回震荡，沿谷底前进极慢。动量把连续几步的梯度做指数加权平均，方向相反的震荡分量相互抵消，方向一致的前进分量不断累积，于是横向摆动被抑制、纵向速度被放大，等效于把条件数调小。这也是动量与预条件在几何上属于同一类操作的原因。

7.2 Nesterov 加速

加速下降法是由俄罗斯数学家尤里·涅斯捷罗夫发明的，因而常以他的名字来命名。加速下降法的思想与标准动量方法类似，唯一的区别在于加速下降法对动量的计算是在施加当前速度之后才进行的：

$$v \leftarrow \mu v - \eta \nabla_\theta J(\theta + \mu v), \qquad \theta \leftarrow \theta + v$$

计算出基于之前的更新得到的动量 v 后，加速下降法先将参数 θ 临时更新为 θ+μv，再在 θ+μv 上计算梯度。与标准动量方法相比，加速下降法可以避免过快更新，因而稳定性更好。

这个"先走一步再看梯度"的做法有直观解释：如果冲过头了，在预测位置上算出的梯度会把参数往回拉，起到提前刹车的作用。

7.3 坐标下降

在很多情况下，"分而治之"是解决优化问题的高效方法，坐标下降法体现的就是这种思想。当优化问题中存在多个自变量时，坐标下降法先保持其他变量不变，针对单一变量 x_1 最小化 f(x)，再针对单一变量 x_2 最小化，依此类推，直到所有变量循环完成。

为了保证对单一变量的优化能够达到全局优化的效果，待优化的目标函数最好是个凸函数，还可以添加 L1 范数的正则化项。此外，每轮优化中随机选取单一变量也可以取得较好的效果。

四个常见误区

- **"深度学习难训是因为局部极小太多"**——高维空间里局部极小的损失值往往接近全局最优，真正拖慢训练的是鞍点与平台区。
- **"SGD 的噪声是缺陷"**——噪声帮助模型跳出尖锐极小、趋向平坦解，反而改善泛化。全批量梯度下降没有这个效果。
- **"动量只是加速"**——它的核心作用是平滑梯度方向、抑制震荡，在狭长山谷里比单纯加速更关键。
- **"自适应学习率一定优于调好的 SGD"**——在不少视觉任务上，精调学习率与动量的 SGD 仍能追平甚至超过自适应方法；选择取决于任务与调参预算。

八、要点回顾

1. 深度网络的损失函数一般非凸，凸优化的局部极小即全局极小的性质不再成立。
2. 病态 Hessian 表现为条件数极大，使梯度下降在平缓方向上进展极慢。
3. 隐藏单元的对称性导致 (n!)^m 个等价极小，但它们的函数值相同，危害有限。
4. 高维空间中鞍点远比局部极小常见，牛顿法因寻找零梯度点而更易受其影响。
5. 随机梯度下降用小批量估计梯度，以更大的估计方差换取计算效率，噪声的期望为零。
6. 学习率随训练衰减，是训练时间与最终精度之间的折中；预热与余弦退火是常用调度。
7. 降噪方法包括动态采样、梯度聚合与迭代平均，前两者可获线性收敛，后者降低结果方差。
8. 二阶方法通过引入曲率信息提升收敛速度，但存储与求逆代价使其难以用于大模型。
9. 动量方法对历史梯度做指数加权平均，抑制震荡、加速沿谷底前进。
10. Nesterov 加速在预测位置上计算梯度，起到提前刹车的作用，稳定性更好。
11. 坐标下降每次只优化一个变量，适用于凸目标且便于加入 L1 正则。

九、名词汇总

名称	英文全称	英文缩写	简短介绍
凸优化	Convex Optimization	—	局部极小即全局极小的优化问题类别
非凸优化	Non-convex Optimization	—	存在多个局部极小的优化问题类别
Hessian 矩阵	Hessian Matrix	—	由二阶偏导数构成的对称矩阵，描述损失曲面曲率
病态矩阵	Ill-conditioned Matrix	—	输入微小扰动会引起解大幅变化的矩阵
条件数	Condition Number	—	最大与最小特征值之比，衡量病态程度
局部极小	Local Minimum	—	邻域内函数值最小的点，未必是全局最优
鞍点	Saddle Point	—	梯度为零但既非极大也非极小的临界点
隐藏单元不可辨认	Unidentifiability of Hidden Units	—	交换隐藏单元及其权重后模型等价的性质
经验风险	Empirical Risk	—	训练集上的平均损失，用于近似期望风险
梯度下降	Gradient Descent	GD	沿负梯度方向迭代更新参数的最优化方法
随机梯度下降	Stochastic Gradient Descent	SGD	用小批量样本估计梯度并更新参数的优化方法
批量大小	Batch Size	—	每次迭代参与梯度估计的样本数目
小批量	Mini-batch	—	介于单样本与全量之间的一次抽样集合
学习率	Learning Rate	—	控制每次参数更新步长的超参数
学习率调度	Learning Rate Schedule	—	随训练进程调整学习率的策略
阶梯衰减	Step Decay	—	在预定轮次把学习率乘以固定系数的调度
余弦退火	Cosine Annealing	—	按余弦曲线把学习率平滑降到接近零的调度
预热	Warmup	—	训练初期让学习率从很小值逐步升到目标值的策略

名称	英文全称	英文缩写	简短介绍
动态采样	Dynamic Sampling	—	逐步增大梯度估计所用子集容量的降噪方法
梯度聚合	Gradient Aggregation	—	对历史梯度估计加权平均以改善搜索方向的方法
迭代平均	Iterate Averaging	—	对历史参数取值求平均以降低方差的降噪方法
牛顿法	Newton's Method	—	用 Hessian 逆矩阵调整步长的二阶优化方法
拟牛顿法	Quasi-Newton Method	—	不显式求 Hessian 逆而近似其逆矩阵的二阶方法
有限内存 BFGS	Limited-memory BFGS	L-BFGS	只保存最近若干步信息的拟牛顿法
高斯牛顿法	Gauss-Newton Method	—	用雅可比矩阵近似 Hessian 的二阶方法
动量	Momentum	—	累积历史梯度形成速度、抑制震荡的加速方法
Nesterov 加速	Nesterov Accelerated Gradient	NAG	在预测位置计算梯度的动量变体
坐标下降	Coordinate Descent	CD	每轮只沿一个坐标方向优化目标的算法
自适应学习率	Adaptive Learning Rate	—	按参数或维度自动调节步长的方法总称